

AI-Powered Meeting Summarizer Using Azure OpenAI API

1. Objectives

- Implement Azure OpenAI's GPT Models: Leverage AzureOpenAI's GPT models to process and summarize meeting transcripts.
- Develop a Functional Application: Create an application that ingests meeting transcripts and produces concise summaries.
- Enhance Productivity: Provide users with quick insights from lengthy meetings, aiding efficient information dissemination.

2. Problem Statement

- In today's fast-paced work environment, attending every meeting or reviewing lengthy meeting notes can be challenging. An AI-powered tool that automatically generates concise summaries from meeting transcripts is invaluable.
- This application aims to process raw meeting transcripts and produce clear, concise summaries highlighting key points, decisions, and action items.

3. Inputs / Shared Artifacts

- Azure OpenAI Resource:
 - API key
 - Endpoint URL
 - Deployment name (to be set as environment variables).

4. Expected Outcomes

- Functional Application: Users can upload meeting transcripts and receive summarized versions.
- User Interface: Simple UI (command-line or web-based) for uploading transcripts and displaying summaries.

5. Concepts Covered

- Using GPT models for text summarization.
- Processing large textual inputs (meeting transcripts).
- Designing user interfaces for file input and result display.
- Integrating Azure OpenAI API in applications.
- Writing clear and concise documentation.

Example: Step-by-Step Instructions with Code Snippet

```
from openai import AzureOpenAI
import os

os.environ["AZURE_OPENAI_ENDPOINT"] = ""
os.environ["AZURE_OPENAI_API_KEY"] = ""
os.environ["AZURE_DEPLOYMENT_NAME"] = "GPT-4o-mini"

api_version = "2024-07-01-preview"
client = AzureOpenAI(
    api_version=api_version,
    azure_endpoint=os.getenv("AZURE_OPENAI_ENDPOINT"),
    api_key=os.getenv("AZURE_OPENAI_API_KEY"),
)

# Step 2: Load transcript text (example: read from a file)
try:
    with open("meeting_transcript.txt", "r") as file:
        transcript = file.read()
except:
    transcript = """[10:02 AM - Project Sync Meeting]
Anna: Chào mọi người, hôm nay mình sẽ cập nhật tiến độ dự án AI chatbot.
Minh: Backend đã xong phần xử lý intent, còn API thì đang test.
Trang: Bên frontend mình gặp chút lỗi khi hiển thị gợi ý từ chatbot, sẽ fix hôm nay.
Anna: Tốt rồi. Nhớ cập nhật trên Jira trước chiều nhé. Có gì blocker không?
Minh: Không có gì lớn.
Trang: Mình cần thêm access từ devops.
Anna: Ok, mình sẽ nhờ Huy cấp cho."""

# Step 3: Craft prompt for summarization

prompt = f"Summarize the following meeting transcript with key points, decisions, and action items:\n\n{transcript}"

# Step 4: Call OpenAI ChatCompletion API

response = client.chat.completions.create(
    model=os.getenv("AZURE_DEPLOYMENT_NAME"),
    messages=[

        {"role": "system", "content": "You are a helpful assistant specialized in summarizing meeting notes."},
        {"role": "user", "content": prompt}
    ],
    temperature=0.3,
    max_tokens=500
)

# Step 5: Extract and display summary
summary = response.choices[0].message.content.strip()
print("Meeting Summary:\n")
print(summary)
```

Final Submission Checklist

- Submit your application code with clear comments in single .py file.
- Use dummy input list (auto input). Do not use manual input (input() built-in).
- Include sample meeting transcript files and example outputs in README or in logs file (5 samples)